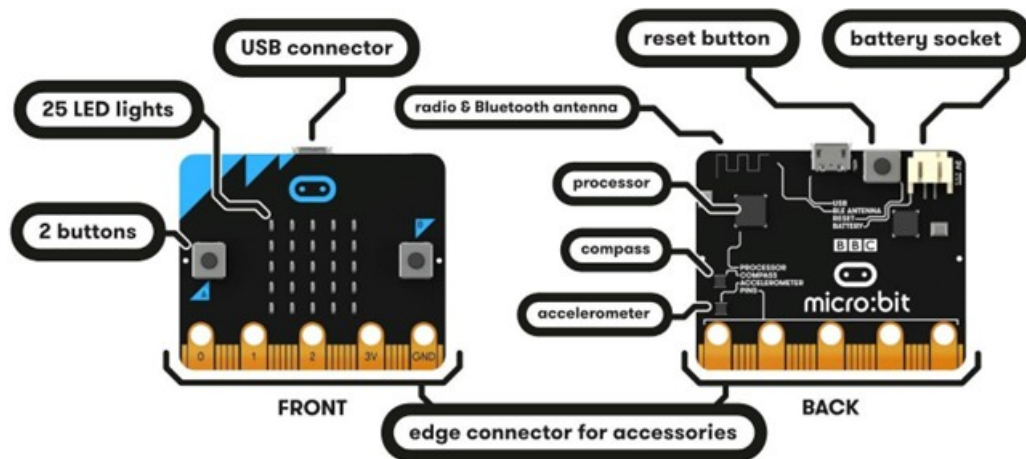


Conocer el Hardware

Empecemos por el principio, el hardware. A continuación podemos ver una representación de la placa con la que vamos a trabajar en este manual.



La micro:bit tiene las siguientes características hardware:

- 25 LED programables individuales
- 2 botones programables
- Pines de conexión física
- Sensores de luz y temperatura
- Sensores de movimiento (acelerómetro y brújula)
- Comunicación inalámbrica, vía radio y Bluetooth
- Interfaz USB

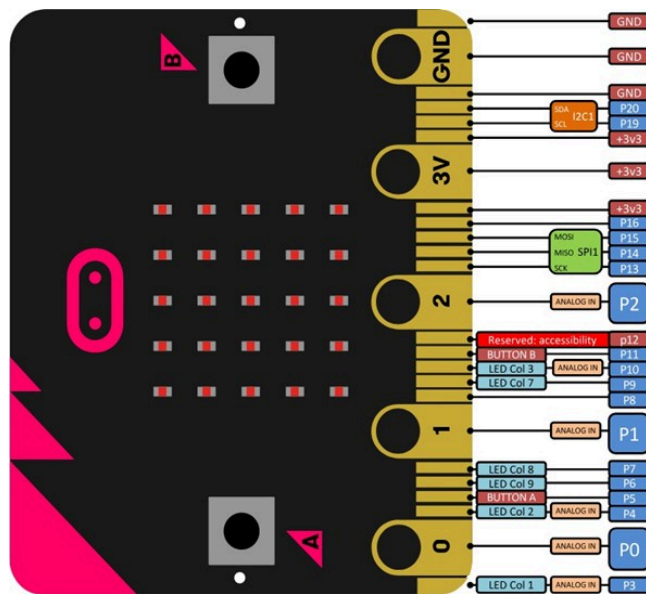
Para más información sobre la micro:bit, podéis visitar la página:

<https://microbit.org/guide/features/>.

No es necesario que los principiantes dominen esta sección, pero sí es necesario tener una idea general. Sin embargo, si queremos ser desarrolladores, la información sobre el hardware resultará muy útil. Encontraremos información detallada sobre el hardware de la micro:bit [aquí](#).

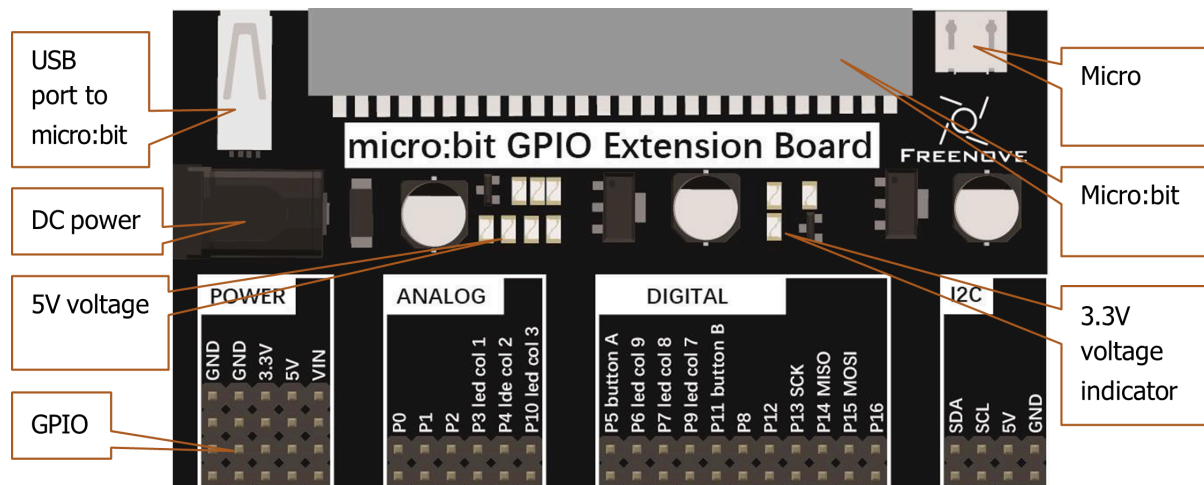
GPIO

La GPIO, es decir, los pines de entrada/salida de uso general, son un componente importante de la micro:bit para conectar dispositivos externos. Todos los sensores y dispositivos de la placa se comunican entre sí a través de los GPIO del micro:bit. A continuación se muestra el diagrama de numeración y funciones:



GPIO - Placa de extensión

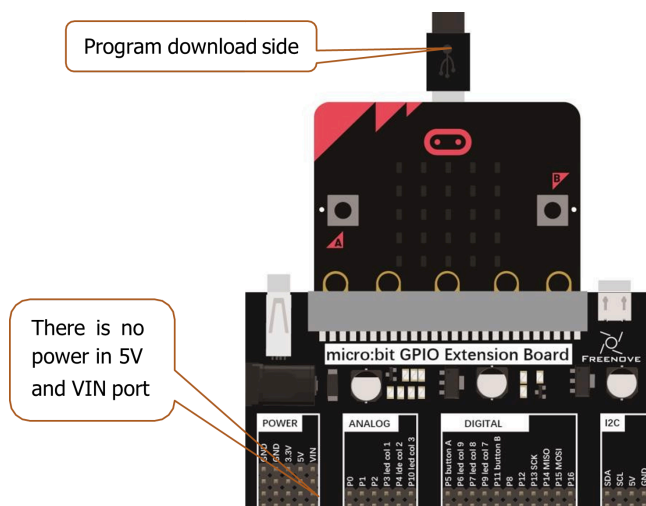
La placa de extensión es un accesorio que permite conectar la micro:bit a otros dispositivos electrónicos conectado a la placa micro:bit a través del socket correspondiente. Esta placa proporciona una forma más fácil de acceder a los pines GPIO de la micro:bit, lo que facilita la conexión de sensores, actuadores y otros componentes electrónicos. A continuación se muestra un ejemplo de una placa de extensión:



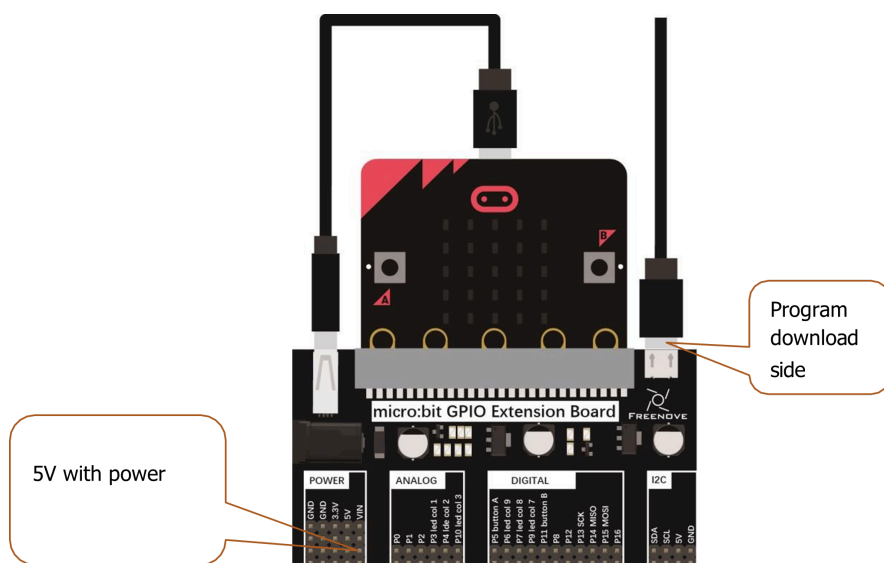
En esta extensión se han añadido puertos de entrada - salida adicionales con voltajes de 5V y VIN(9V) para suplir los requerimientos de algunos dispositivos.

Cómo usar la extensión

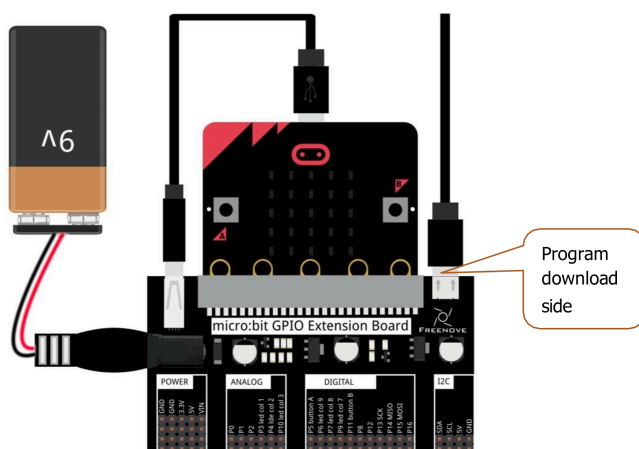
Si no es necesario utilizar los voltajes extendidos podemos hacer uso de la placa de la siguiente manera:



Si los periféricos necesitan 5V, pero no mucha potencia, la siguiente imagen muestra cómo hacerlo:



Por último, si las necesidades son elevadas, el diagrama siguiente muestra la configuración adecuada:



Programación de la Micro::bit 2

Con este manual vamos a aprender a programar la Micro::bit 2, un microcontrolador diseñado para la educación y la experimentación en electrónica y programación. La Micro::bit 2 cuenta con una variedad de sensores, botones, una pantalla LED y conectividad Bluetooth, lo que la hace ideal para proyectos interactivos. Utilizaremos como base el lenguaje Rust, que es un lenguaje de programación moderno y seguro, para crear programas que interactúen con los componentes de la Micro::bit 2.

Referencias

Antes de nada unas pocas referencias:

- [Documentación oficial de la Micro::bit 2](#)
- [Documentación de Rust](#)
- [micro::bit v2 Embedded Discovery Book \(castellano\)](#)
- [Rust Embedded Book](#)

Instalación del entorno de desarrollo

Para programar la Micro::bit 2 con Rust, necesitamos configurar nuestro entorno de desarrollo. Antes de pasar a la instalación real, enumeraremos los requisitos que he probado en este manual:

- Rust 1.79.0 o una toolchain más reciente.
- `gdb-multiarch`. Herramienta de depuración. La versión más antigua que hemos probado es la 10.2, pero es probable que otras versiones también funcionen. Si la distribución/plataforma no tiene `gdb-multiarch` disponible, podemos usar `arm-none-eabi-gdb` para depurar. Además, algunos binarios de `gdb` están contruidos con capacidades multiplataforma: se puede encontrar más información sobre esto en el capítulo de depuración de este libro.
- `[ cargo-binutils ]`. Versión 0.3.6 o posterior. `[ cargo-binutils ]`: <https://github.com/rust-embedded/cargo-binutils>
- `[ probe-rs-tools ]`. Versión 0.24.0 o más reciente. `[ probe-rs-tools ]`: <https://probe.rs/docs/overview/about-probe-rs/>
- `minicom` en Linux y macOS. Se ha probado la versión: 2.7.1. Esperamos que versiones posteriores también funcionen.
- `PuTTY` en Windows.

Instalación de Rust

Para instalar Rust, podemos utilizar `rustup`, que es la herramienta oficial para gestionar las versiones de Rust. Si no tenemos `rustup` instalado, seguiremos las instrucciones en la [página oficial de Rust](#).

Instrucciones de la página: Para empezar a usar Rust, descarga el instalador, ejecuta el programa y sigue las instrucciones que aparecen en pantalla. Es posible que tengas que instalar las herramientas de compilación de Visual Studio C++ cuando se solicite. Si no utilizas Windows, consulta "Otros métodos de instalación".

```
$ rustc -V
rustc 1.94.0 (4a4ef493e 2026-03-02)
```

Una vez instalado `rustup`, tenemos que instalar la toolchain de Rust con el comando:

```
$ rustup target add thumbv7em-none-eabihf
```

Solo hay que hacerlo una vez; `rustup` actualizará automáticamente la cadena de compilación si se lo pedimos. Explicaremos más adelante por qué es necesario instalar esta toolchain específica.

Instalación bajo Windows

La mayoría de ordenadores educativos utilizan Windows, por lo que vamos a detallar los pasos para instalar Rust en este sistema operativo. Si estamos usando otra plataforma recomiendo leer el libro de micro::bit v2 Embedded Discovery Book.

`gdb-multiarch (arm-none-eabi-gdb)`

Hemos descargado e instalado el siguiente archivo de la página oficial de Arm: [Aquí](#)

Arm proporciona ejecutables (`.exe`) para Windows. Se pueden encontrar en [gcc](#), solo hay que seguir las instrucciones. Justo antes de que finalice el proceso de instalación, hay que marcar/seleccionar la opción “Añadir ruta a la variable de entorno”. Si no aparece dicha opción se tendrá que hacer desde el diálogo del sistema y añadir la siguiente ruta para la instalación por defecto:

```
C:\Program Files\Arm\GNU Toolchain mingw-w64-x86_64-arm-none-eabi\bin
```

A continuación, comprobaremos que las herramientas se encuentran en el `%PATH%`:

```
$ arm-none-eabi-gcc -v
(..)
gcc version 15.2.1 20251203 (Arm GNU Toolchain 15.2.Rel1 (Build arm-15.86))
```

`cargo-binutils`

```
$ rustup component add llvm-tools
$ cargo install cargo-binutils --vers '^0.4'
$ cargo size --version
cargo-size 0.4.0
```

`probe-rs-tools`

NOTA Si existen en el sistema versiones anteriores de `probe-run`, `probe-rs` o `cargo-embed` instaladas, hay que eliminarlas antes de seguir adelante, ya que podrían causar problemas. En particular, `probe-run` ya no existe oficialmente. Hay que eliminarlas si es necesario:

```
$ cargo uninstall cargo-embed
$ cargo uninstall probe-run
$ cargo uninstall probe-rs
$ cargo uninstall probe-rs-cli
```

Para instalar `probe-rs-tools`, hay que seguir las instrucciones de la página oficial en <https://probe.rs>. En mi caso la instalación mediante cargo no me ha dado ningún tipo de problema.

```
$ cargo install probe-rs-tools
```

```
Finished `release` profile [optimized] target(s) in 3m 34s
Installing C:\Users\arcipreste\.cargo\bin\cargo-embed.exe
Installing C:\Users\arcipreste\.cargo\bin\cargo-flash.exe
Installing C:\Users\arcipreste\.cargo\bin\probe-rs.exe
Installed package `probe-rs-tools v0.31.0` (executables `cargo-embed.exe`, `cargo-
flash.exe`, `probe-rs.exe`)
```

```
C:\Users\...>probe-rs
The probe-rs CLI
```

Instalar la herramienta `probe-rs-tools` configurará en nuestro ordenador diversas herramientas muy útiles, incluyendo `probe-rs` y `cargo-embed` (que normalmente se ejecutan como un comando de Cargo). Debemos comprobar que todo funciona correctamente antes de pasar a la siguiente sección.

```
$ cargo embed --version
cargo embed 0.31.0 (git commit: crates.io)
```

PuTTY

Se puede descargar `putty.exe` desde [aquí](#) y añadirlo al `%PATH%`.

Primer ejemplo: “Hola, mundo”

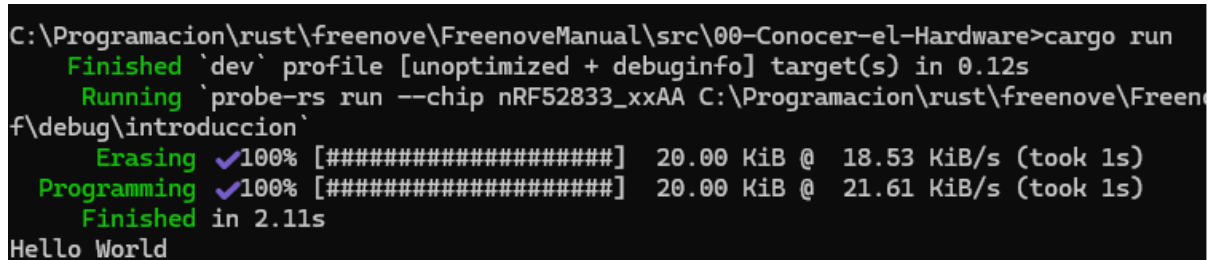
Vamos a crear nuestro primer programa para la Micro::bit 2, que mostrará el mensaje “Hola, mundo” en el terminal. Para ello, seguiremos los siguientes pasos:

- Conectamos la placa a un puerto USB de nuestro ordenador. Comprobamos que se abre una ventana con el contenido de la MB2.
- Abrimos un terminal y nos situamos en el directorio del capítulo inicial:

```
$ cd src/00-Conocer-el-Hardware/
```

- Ejecutamos el siguiente comando para compilar y ejecutar el programa:

```
$ cargo run
```



```
C:\Programacion\rust\freenove\FreenoveManual\src\00-Conocer-el-Hardware>cargo run
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.12s
Running `probe-rs run --chip nRF52833_xxAA C:\Programacion\rust\freenove\FreenoveManual\src\00-Conocer-el-Hardware\src\main.rs`
Erasing ✓100% [#####] 20.00 KiB @ 18.53 KiB/s (took 1s)
Programming ✓100% [#####] 20.00 KiB @ 21.61 KiB/s (took 1s)
Finished in 2.11s
Hello World
```

El código fuente del programa se encuentra en el archivo `src/main.rs`. A continuación, se muestra el contenido del archivo:

```
#![no_main]
#![no_std]

use cortex_m::asm::{nop};
use nrf52833_pac as _;
use panic_rtt_target as _;
use rtt_target::{rprintln, rtt_init_print};

use cortex_m_rt::entry;

#[entry]
fn main() -> ! {
    rtt_init_print!();
    rprintln!("Hello World");
    loop {
        nop();
    }
}
```

Un poco más de programación

Comprendiendo el primer programa

Vamos a echar un vistazo a nuestro primer programa. Comprobemos el fichero `src/main.rs`:

```
#![no_main]
#![no_std]

use cortex_m::asm::{nop};
use nrf52833_pac as _;
use panic_rtt_target as _;
use rtt_target::{rprintln, rtt_init_print};

use cortex_m_rt::entry;

#[entry]
fn main() -> ! {
    rtt_init_print!();
    rprintln!("Hello World");
    loop {
        nop();
    }
}
```

Los programas para microcontroladores son diferentes de los programas estándar en dos aspectos: `#![no_std]` y `#![no_main]`.

El atributo `no_std` indica que este programa no usará la biblioteca estándar de Rust, la cual asume un sistema operativo subyacente; el programa utilizará en su lugar el crate `core`, un subconjunto de `std` que puede ejecutarse en sistemas hardware directamente.

El atributo `no_main` indica que este programa no usará la interfaz estándar 'main', que está diseñada para aplicaciones de línea de comandos que reciben argumentos. En lugar del 'main' estándar, usaremos el atributo 'entry' del crate `cortex-m-rt` para definir un punto de entrada personalizado. En este programa hemos definido el punto de entrada como `main`, pero se podría haber usado cualquier otro nombre. La función del punto de entrada debe tener la firma `fn() -> !`; este tipo indica que la función no termina. Significa que el programa nunca finaliza: si el compilador detecta que esto sería posible, se negará a compilarlo.

Si observamos con cuidado el directorio del proyecto notaremos que hay un directorio `.cargo` posiblemente oculto en el proyecto. Este directorio contiene un archivo de configuración de Cargo `.cargo/config.toml`.

```
[build]
target = "thumbv7em-none-eabihf"

[target.thumbv7em-none-eabihf]
runner = "probe-rs run --chip nRF52833-xxAA"
rustflags = [
    "-C", "linker=rust-lld",
]
```

Este fichero modifica el proceso de enlace para adaptar la disposición de memoria del programa a los requisitos del dispositivo de desarrollo. Este proceso de enlace es un requisito del crate `cortex-m-rt`. El archivo `.cargo/config.toml` también le dice a Cargo cómo construir y ejecutar el código en nuestra MB2.

Hay también un fichero `Embed.toml`:


```

[default.probe]
protocol = "Swd"

[default.general]
chip = "nrf52833_xxAA"

[default.reset]
halt_afterwards = true

[default.rtt]
enabled = true

[default.gdb]
enabled = false

```

Este fichero informa a `cargo-embed` que:

- Trabaja con un chip NRF52833.
- Queremos detener la ejecución en el chip después de flashearlos, por lo que nuestro programa se detiene antes de `main`.
- Queremos deshabilitar/habilitar RTT. RTT es un protocolo que permite al chip enviar texto a un depurador. Ya has visto RTT en acción fue la primera versión del programa inicial.
- Queremos Deshabilitar/habilitar GDB. Este paso es necesario para procesos de depuración tal y como veremos al final de este capítulo.

Generando el binario

El primer paso es construir el binario. Dado que el microcontrolador tiene una arquitectura diferente a la de nuestro ordenador, tendremos que realizar una compilación cruzada. Hacerlo en Rust es tan simple como pasar un flag extra `--target` a `rustc` o Cargo. La parte complicada es averiguar el argumento de ese flag: el *nombre* del destino.

Como ya hemos visto, el microcontrolador del micro:bit tiene un procesador Cortex-M4F. `rustc` sabe cómo compilar de forma cruzada para la arquitectura Cortex-M y proporciona varios destinos que cubren las diferentes familias de procesadores dentro de esa arquitectura:

- `thumbv6m-none-eabi`, para los procesadores Cortex-M0 y Cortex-M1
- `thumbv7m-none-eabi`, para el procesador Cortex-M3
- `thumbv7em-none-eabi`, para los procesadores Cortex-M4 y Cortex-M7
- `thumbv7em-none-eabihf`, para los procesadores Cortex-M4F y Cortex-M7F
- `thumbv8m.main-none-eabi`, para los procesadores Cortex-M33 y Cortex-M35P
- `thumbv8m.main-none-eabihf`, para los procesadores Cortex-M33F y Cortex-M35PF

“Thumb” aquí se refiere a una versión del conjunto de instrucciones Arm que tiene instrucciones más pequeñas para reducir el tamaño del código. La denominación `hf / F` significa que implementa aceleración de punto flotante por hardware.

Para la MB2, micro:bit v2, queremos el destino `thumbv7em-none-eabihf`.

Antes de realizar la compilación cruzada, es necesario descargar una versión precompilada de la biblioteca estándar (en realidad, una versión reducida de ella) en el host local. Se hace usando `rustup`:

```
$ rustup target add thumbv7em-none-eabihf
```

Solo hay que hacerlo una vez; `rustup` actualizará este destino (reinstalando un nuevo componente de la biblioteca estándar `rust-std` que contiene la biblioteca `core` que usamos) cada vez que actualicemos la cadena de compilación. Por lo tanto, **se puede omitir este paso si ya se agregó la `toolchain` necesaria anteriormente.**

Con el componente `rust-std` en su lugar, ya es posible compilar el programa de forma cruzada usando Cargo. Nos aseguraremos de estar en el directorio `src/00-Conocer-el-Sistema`, luego lo construimos. Este código inicial es un ejemplo, así que lo compilamos como tal.

```
$ cargo build
  Compiling semver-parser v0.7.0
  Compiling proc-macro2 v1.0.86
  ...

Finished dev [unoptimized + debuginfo] target(s) in 33.67s
```

Ok, ya tenemos un ejecutable. ¡No hará mucho!, imprime el mensaje “Hello, World!”.

Flashearlo

Flashear es el proceso de mover el programa a la memoria del microcontrolador. Una vez hecho, el microcontrolador ejecutará el programa cada vez que se encienda.

El programa será el único existente en la memoria. Por esto me refiero a que no hay nada más ejecutándose en el microcontrolador: ni un sistema operativo, ni un “daemon”, nada. Nuestro programa tiene control total sobre el dispositivo.

Pasarlo al microcontrolador es muy simple, gracias a `cargo embed\run`.

```
$ cargo run
(...)
Erasing sectors ✓ [00:00:00]
[#####] 2.00KiB/ 2.00KiB @
4.21KiB/s (eta 0s )
Programming pages ✓ [00:00:00]
[#####] 2.00KiB/ 2.00KiB @
2.71KiB/s (eta 0s )
Finished flashing in 0.608s
```

Es importante fijarse que si queremos depurar se tiene que lanzar con `cargo embed`, y de esta forma no termina después de mostrar la última línea. Esto es intencionado: no hay que cerrar `cargo embed`, ya que lo necesitamos en este estado para depurar.

Depuración de programas

La depuración de programas es una parte fundamental del desarrollo de software, especialmente cuando se trabaja con microcontroladores como la Micro::bit. Para un funcionamiento correcto de la depuración, necesitamos configurar el proyecto para que utilice `gdb` en lugar de `rtt`. Para ello, editamos el archivo `Embed.toml` y establecemos `enabled = false` para `rtt` y `enabled = true` para `gdb`. El contenido del archivo debe ser el siguiente:

```
[default.rtt]
enabled = false

[default.gdb]
enabled = true
```

Tras modificar el fichero `Embed.toml` en el directorio `00-Conocer-el-Sistema`, ejecutamos pero lanzando la depuración con `cargo embed`:

```
$ cargo embed
```

Nos conectamos a la sesión de depuración con `gdb` desde otra consola (recordar que solo se muestran ejemplos para el SSOO Windows). Para ello, abrimos otra terminal y nos situamos en la raíz del proyecto. Ejecutamos `arm-none-eabi-gdb` con el binario que queremos depurar como argumento:

```
$ cd raiz_del_libro
$ arm-none-eabi-gdb ./target/thumbv7em-none-eabihf/debug/introduccion
```

Si nos da un fallo donde no encuentra el binario, recordar que el directorio `target` se encuentra en la raíz del proyecto, no en `src/00-Conocer-el-Sistema`. Ya dentro de `gdb` nos unimos a la sesión de depuración remota con el comando `target remote` y el puerto que vemos al ejecutar con `cargo embed` (en este caso, el puerto 1337):

```
(gdb) target remote :1337
Remote debugging using :1337
```

Los puntos de ruptura se pueden usar para detener el flujo normal de un programa. El comando `continue` permitirá que el binario se ejecute libremente *hasta* que alcance un punto de ruptura. En este caso, hasta que alcance la función `main` porque hemos establecido uno allí.

```
(gdb) break main
...
(gdb) continue
...
```

Para ver el contenido de las variables, podemos usar el comando `print` seguido del nombre de la variable. Por ejemplo, para ver el contenido de la variable `x`:

```
(gdb) print x
$1 = 0
```

Si queremos ejecutar el programa paso a paso, podemos usar el comando `next` para avanzar una línea de código a la vez. Esto nos permite observar cómo cambian las variables y el flujo del programa en tiempo real.

```
(gdb) next
16          loop {}
```

El comando `monitor reset` resetea el microcontrolador y lo para en el punto de entrada.

```
(gdb) monitor reset
```

Ya hemos terminado la sesión de depuración. Podemos salir con el comando `quit`.

```
(gdb) quit
A debugging session is active.
```

```
Inferior 1 [Remote target] will be detached.
```

```
Quit anyway? (y or n) y
Detaching from program: $PWD/target/thumbv7em-none-eabihf/debug/meet-your-software,
Remote target
Ending remote debugging.
[Inferior 1 (Remote target) detached]
```

Resumen

En este capítulo hemos conocido la máquina y como programarla. Hemos aprendido a configurar el SSOO para ejecutar Rust y en concreto programar sistemas embebidos, hemos configurado la toolchain adecuada para la MB2 de Rust y hemos aprendido a compilar, flashear y ejecutar. Además, hemos visto cómo utilizar el depurador para analizar el comportamiento de nuestros programas. En resumen, hemos adquirido los conocimientos necesarios para desarrollar software en sistemas embebidos utilizando Rust.

A partir de ahora, avanzaremos hacia la programación de sistemas embebidos más complejos, a través de proyectos específicos. En concreto cada capítulo se dedicará a una parte del hardware específica.

Para terminar

Hay un tema que no hemos tratado, el uso de IDEs específicos. Si bien Rust no requiere un IDE para programar, existen algunos que pueden facilitar el desarrollo, como Visual Studio Code o RustRover. Cualquiera de los dos puede ser utilizado para programar en Rust, y ambos ofrecen características como ia, autocompletado, depuración y gestión de proyectos que pueden mejorar la productividad. Sin embargo, es importante recordar que el conocimiento de la línea de comandos y las herramientas básicas sigue siendo fundamental para un desarrollador de sistemas embebidos.

Como configurar IntelliJ o RustRover

Al editar la configuración de compilación de RustRover, estos son algunos valores no predeterminados:

- Hay que modificar el comando. Cuando se indique que hay que ejecutar `cargo embed FLAGS`, se deberá cambiar el valor predeterminado `run` por el comando `embed FLAGS` si queremos depurar, si solo lo lanzamos no es necesario.
- Se tiene que activar la opción “Emular terminal en la consola de salida”. De lo contrario, el programa no podrá mostrar texto en un terminal.
- Nos aseguraremos que el directorio de trabajo sea `./src/N-nombre_capitulo`, siendo `N-nombre_capitulo` el directorio del capítulo que estamos leyendo. No se puede ejecutar desde el directorio `src 0 FreenoveManual`.

Capítulo 1 - Matriz de LED

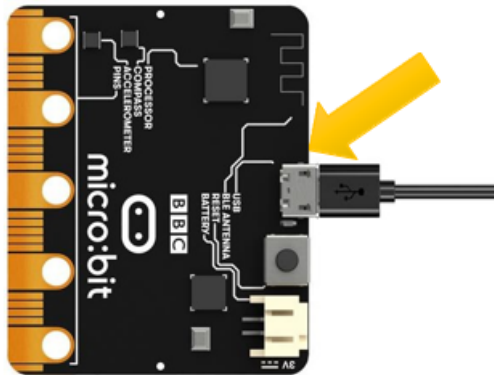
Descripción del proyecto 1.1

Este proyecto tiene como objetivo crear una animación en la matriz de LEDs de un corazon parpadeando.

Hardware necesario

- Micro:bit
- Micro Usb

Esquema de conexión



Código fuente

```
#![no_main]
#![no_std]

use cortex_m_rt::entry;
use embedded_hal::delay::DelayNs;
use microbit::{board::Board, display::blocking::Display, hal::Timer};
use panic_rtt_target as _;
use rtt_target::rtt_init_print;

#[entry]
fn main() -> ! {
    rtt_init_print!();

    let board = Board::take().unwrap();
    let mut timer = Timer::new(board.TIMER0);
    let mut display = Display::new(board.display_pins);
    let light_heart_big = [
        [0, 1, 0, 1, 0],
        [1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1],
        [0, 1, 1, 1, 0],
        [0, 0, 1, 0, 0],
    ];
    let light_heart_small = [
        [0, 1, 0, 1, 0],
        [1, 0, 1, 0, 1],
        [1, 0, 0, 0, 1],
        [0, 1, 0, 1, 0],
        [0, 0, 1, 0, 0],
    ];

    loop {
        display.show(&mut timer, light_heart_big, 1000);
        timer.delay_ms(1000_u32);
        display.show(&mut timer, light_heart_small, 1000);
        timer.delay_ms(1000_u32);
        // display.clear();
    }
}
```

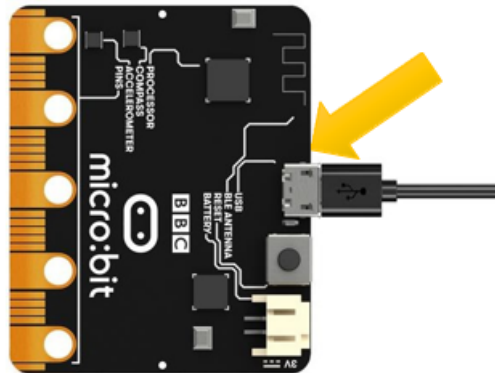
Descripción del proyecto 1.2

Se pretende desplazar el corazón del segundo frame del ejercicio anterior fuera de la matriz de LEDs.

Hardware necesario

- Micro:bit
- Micro Usb

Esquema de conexión



Código fuente

```
#![no_main]
#![no_std]

use cortex_m_rt::entry;
use embedded_hal::delay::DelayNs;
use microbit::{board::Board, display::blocking::Display, hal::Timer};
use panic_rtt_target as _;
use rtt_target::rtt_init_print;

include!("lib_cap.rs");

#[entry]
fn main() -> ! {
    rtt_init_print!();

    let board = Board::take().unwrap();
    let mut timer = Timer::new(board.TIMER0);
    let mut display = Display::new(board.display_pins);
    let light_heart_big = [
        [0, 1, 0, 1, 0],
        [1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1],
        [0, 1, 1, 1, 0],
        [0, 0, 1, 0, 0],
    ];
    let mut light_heart_small = [
        [0, 1, 0, 1, 0],
        [1, 0, 1, 0, 1],
        [1, 0, 0, 0, 1],
        [0, 1, 0, 1, 0],
        [0, 0, 1, 0, 0],
    ];

    loop {
        display.show(&mut timer, light_heart_big, 1000);
        timer.delay_ms(1000_u32);
        for _ in 0..=5 {
            display.show(&mut timer, light_heart_small, 500);
            timer.delay_ms(10_u32);
            lib_cap::rotate_column_matrix(&mut light_heart_small);
        }
    }
}
```

Comando de ejecución

```
$cargo run --example heart_move
```

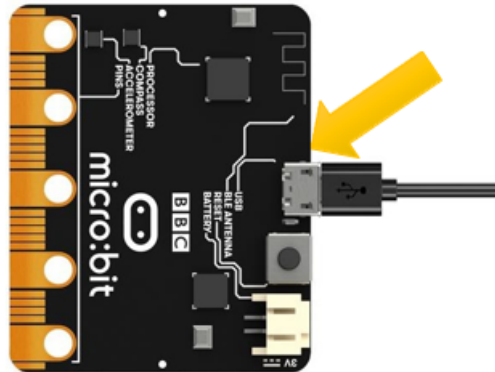
Descripción del proyecto 1.3

Se pretende mostrar una secuencia de números en la matriz de LEDs, donde cada número se mostrará durante un segundo antes de pasar al siguiente.

Hardware necesario

- Micro:bit
- Micro Usb

Esquema de conexión



Código fuente

```
#![no_main]
#![no_std]

use cortex_m_rt::entry;
use embedded_hal::delay::DelayNs;
use microbit::{board::Board, display::blocking::Display, hal::Timer};
use panic_rtt_target as _;
use rtt_target::rtt_init_print;

include!("lib_cap.rs");

#[entry]
fn main() -> ! {
    rtt_init_print!();

    let board = Board::take().unwrap();
    let mut timer = Timer::new(board.TIMER0);
    let mut display = Display::new(board.display_pins);

    let mut light_heart_small = [[0; 5];5];
    let data = [1,2,0,5,1];

    loop {
        for num in data {
            lib_cap::number_to_display(num, &mut light_heart_small);
            display.show(&mut timer, light_heart_small, 1000);
            timer.delay_ms(500_u32);
        }
    }
}
```


Comando de ejecución

```
$cargo run --example numbers
```

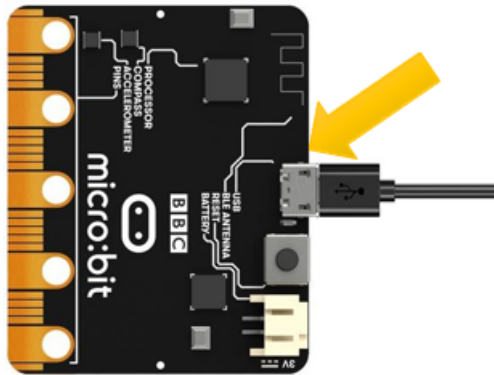
Descripción del proyecto 1.4

Se pretende mostrar un conjunto de números deslizándose por la matriz de LEDS.

Hardware necesario

- Micro:bit
- Micro Usb

Esquema de conexión



Código fuente

```
#![no_main]
#![no_std]

use cortex_m_rt::entry;
use embedded_hal::delay::DelayNs;
use microbit::{board::Board, display::blocking::Display, hal::Timer};
use panic_rtt_target as _;
use rtt_target::rtt_init_print;

include!("lib_cap.rs");

#[entry]
fn main() -> ! {
    rtt_init_print!();

    let board = Board::take().unwrap();
    let mut timer = Timer::new(board.TIMER0);
    let mut display = Display::new(board.display_pins);

    let mut light_heart_small = [[0; 5];5];
    let data = "0123";

    loop {
        for num in data.chars() {
            lib_cap::number_to_display(num.to_digit(10).unwrap() as u8, &mut
light_heart_small);
            display.show(&mut timer, light_heart_small, 1000);
            for _ in 0..=5 {
                display.show(&mut timer, light_heart_small, 500);
                timer.delay_ms(10_u32);
                lib_cap::rotate_column_matrix(&mut light_heart_small);
            }
        }
    }
}
```

Comando de ejecución

```
$cargo run --example text_scroll
```

Código fuente de ayuda

```
mod lib_cap{
  pub fn number_to_display(number:u8 ,matriz:&mut [[u8;5];5]){
    match number{
      0 =>
        *matriz = [
          [0, 1, 1, 1, 0],
          [0, 1, 0, 1, 0],
          [0, 1, 0, 1, 0],
          [0, 1, 0, 1, 0],
          [0, 1, 1, 1, 0]],
      1 => {
        *matriz = [
          [0, 0, 1, 1, 0],
          [0, 1, 0, 1, 0],
          [0, 0, 0, 1, 0],
          [0, 0, 0, 1, 0],
          [0, 0, 0, 1, 0]];
      },
      2 => {
        *matriz = [
          [0, 1, 1, 1, 0],
          [0, 0, 0, 1, 0],
          [0, 0, 1, 0, 0],
          [0, 1, 0, 0, 0],
          [0, 1, 1, 1, 0]];
      },
      _ => {
        *matriz = [
          [0, 0, 0, 0, 0],
          [0, 0, 0, 0, 0],
          [1, 1, 1, 1, 1],
          [0, 0, 0, 0, 0],
          [0, 0, 0, 0, 0]];
      }
    }
  }
}

pub fn rotate_column_matrix(matriz:&mut [[u8;5];5]){
  for col in (1..5).rev() {
    matriz[0][col] = matriz[0][col-1];
    matriz[1][col] = matriz[1][col-1];
    matriz[2][col] = matriz[2][col-1];
    matriz[3][col] = matriz[3][col-1];
    matriz[4][col] = matriz[4][col-1];
  }
  {
    matriz[0][0] = 0;
    matriz[1][0] = 0;
    matriz[2][0] = 0;
    matriz[3][0] = 0;
    matriz[4][0] = 0;
  }
}
```

Capítulo 2 - Botones

Los teclados o botones son herramientas importantes para la interacción nosotros y el ordenador. A menudo utilizamos los teclados para introducir texto, escribir comandos, controlar dispositivos, etc. La micro:bit incorpora dos botones programables, A y B, que permiten controlarla para que realice acciones.

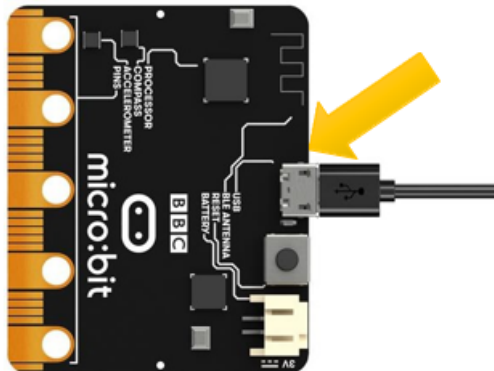
Descripción del proyecto 2.1

Este proyecto tiene como objetivo mostrar dos patrones diferentes en función del botón que se pulse. Si se pulsa el botón A, se mostrará un patrón de flecha hacia la izquierda en la pantalla LED de la MB2. Si se pulsa el botón B, se mostrará un patrón de flecha hacia la derecha.

Hardware necesario

- Micro:bit
- Micro Usb

Esquema de conexión



Código fuente

```
#![no_main]
#![no_std]

use cortex_m_rt::entry;
// use embedded_hal::delay::DelayNs;
use embedded_hal::digital::InputPin;
use microbit::{board::Board, display::blocking::Display, hal::Timer};
use panic_rtt_target as _;
use rtt_target::rtt_init_print;

#[entry]
fn main() -> ! {
    rtt_init_print!();

    let board = Board::take().unwrap();
    let mut display = Display::new(board.display_pins);
    let mut btn_a = board.buttons.button_a;
    let mut btn_b = board.buttons.button_b;
    let mut timer = Timer::new(board.TIMER0);

    let derecha = [
        [0, 0, 1, 0, 0],
        [0, 0, 0, 1, 0],
        [1, 1, 1, 1, 1],
        [0, 0, 0, 1, 0],
        [0, 0, 1, 0, 0],
    ];
    let izquierda = [
        [0, 0, 1, 0, 0],
        [0, 1, 0, 0, 0],
        [1, 1, 1, 1, 1],
        [0, 1, 0, 0, 0],
        [0, 0, 1, 0, 0],
    ];

    loop {
        if btn_a.is_low().unwrap() {
            display.show(&mut timer, izquierda, 1000);
        } else if btn_b.is_low().unwrap() {
            display.show(&mut timer, derecha, 1000);
        }
    }
}
```

Capítulo 3 - Led

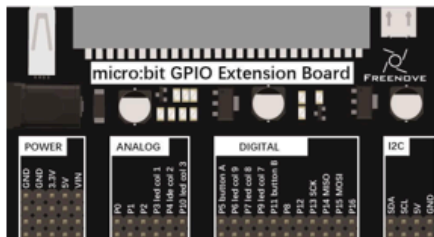
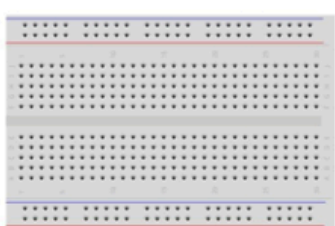
En esta sección aprenderemos a controlar los LEDs de la MB2.

Descripción del proyecto 3.1

Este proyecto tiene como objetivo de hacer parpadear un único LED externo.

Hardware necesario

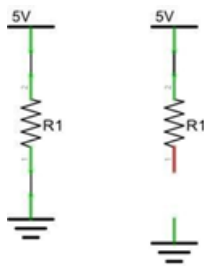
- Micro:bit
- Micro Usb
- Placa de Extensión
- Placa de conexión
- Cable de conexión
- Resistencia de 220 ohmios
- Un Led

Micro bit x1	Expansion board x1	
		
Break board x1	USB cable x1	
		
Jumper F/M x2	Resistor 220Ω x1	LED x1
		

Conociendo los componentes

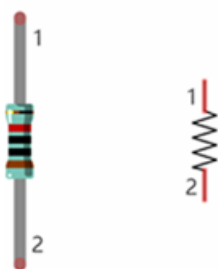
Circuito integrado micro:bit

La unidad de intensidad (I) es el amperio (A). $1\text{ A} = 1\,000\text{ mA}$, $1\text{ mA} = 1\,000\text{ }\mu\text{A}$. Para que haya intensidad, es necesario un circuito cerrado formado por componentes electrónicos. En la siguiente figura: a la izquierda hay un circuito cerrado, por lo que la intensidad circula por el circuito. A la derecha no hay un circuito cerrado, por lo que no hay intensidad.



Resistencias

Las resistencias utilizan el ohmio (Ω) como unidad de medida de su resistencia (R). $1 \text{ M}\Omega = 1\,000 \text{ k}\Omega$, $1 \text{ k}\Omega = 1\,000 \Omega$. Una resistencia es un componente eléctrico pasivo que limita o regula el flujo de corriente en un circuito electrónico. A la izquierda vemos una representación física de una resistencia, y a la derecha está el símbolo que se utiliza para indicar la presencia de una resistencia en un diagrama de circuito o esquema.



Las bandas de color de una resistencia constituyen un código abreviado que se utiliza para identificar su valor de resistencia. Para obtener más detalles sobre los códigos de colores de las resistencias, consulta la ficha incluida en el paquete del kit.

Con una tensión fija, cuanto mayor sea la resistencia añadida al circuito, menor será la corriente de salida. La relación entre corriente, tensión y resistencia se puede expresar mediante esta fórmula: $I = V/R$, conocida como la ley de Ohm, donde I = corriente, V = tensión y R = resistencia. Si se conocen los valores de dos de estos tres parámetros, se puede calcular el valor del tercero. En el siguiente diagrama, la corriente que atraviesa $R1$ es: $I = V/R = 5 \text{ V} / 10 \text{ k}\Omega = 0,0005 \text{ A} = 0,5 \text{ mA}$.



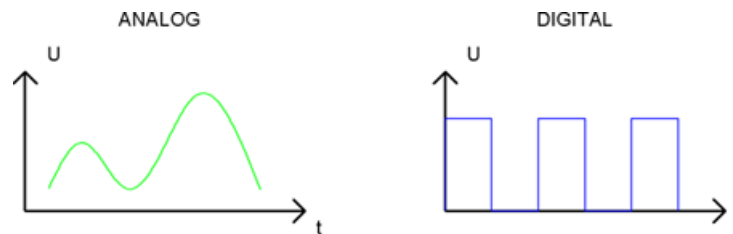
ADVERTENCIA: Nunca conectes los dos polos de una fuente de alimentación con ningún elemento de baja resistencia (por ejemplo, un objeto metálico o un cable pelado), ya que esto provoca un cortocircuito y genera una corriente elevada que puede dañar la fuente de alimentación y los componentes electrónicos.

Nota: A diferencia de los LEDs y los diodos, las resistencias no tienen polos y son no polares (no importa en qué dirección se inserten en un circuito, funcionarán igual).

Señal Analógica y Digital

Una señal analógica es una señal continua tanto en el tiempo como en su valor. Por el contrario, una señal digital o de tiempo discreto es una serie temporal formada por una secuencia de magnitudes.

La mayoría de las señales que encontramos en la vida cotidiana son señales analógicas. Un ejemplo conocido de señal analógica sería cómo la temperatura varía continuamente a lo largo del día y no podría cambiar de forma repentina e instantánea de 0°C a 10°C . Sin embargo, el valor de las señales digitales puede cambiar instantáneamente. Este cambio se expresa numéricamente mediante los números 1 y 0 (la base del código binario). Sus diferencias se aprecian más fácilmente al compararlas en un gráfico como el que se muestra a continuación.

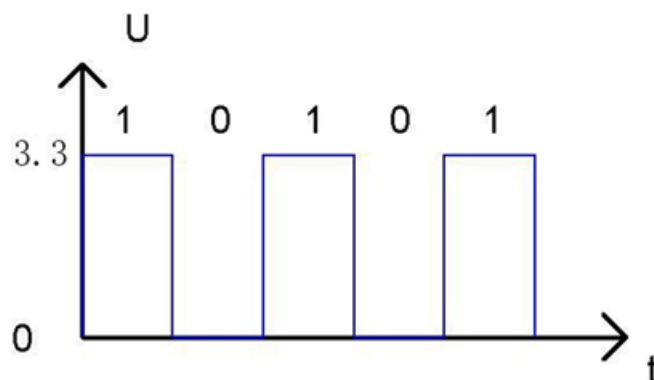


Ten en cuenta que las señales analógicas son ondas curvas y las señales digitales son “ondas cuadradas”. En aplicaciones prácticas, solemos utilizar el sistema binario como señal digital, es decir, una secuencia de ceros y unos. Dado que una señal binaria solo tiene dos valores (0 o 1), ofrece una gran estabilidad y fiabilidad. Por último, tanto las señales analógicas como las digitales pueden convertirse unas en otras.

Nivel bajo y alto de tensión

En los circuitos, los valores binarios (0 y 1) se representan como nivel bajo y nivel alto. El nivel bajo suele ser igual a la tensión de tierra (0 V). El nivel alto suele ser igual a la tensión de funcionamiento de los componentes.

El nivel bajo del Micro:bit es de 0 V y el nivel alto es de $3,3\text{ V}$, tal y como se muestra a continuación. Cuando el puerto de E/S del Micro:bit emite un nivel alto, se pueden controlar directamente componentes de bajo consumo, como los LEDs.



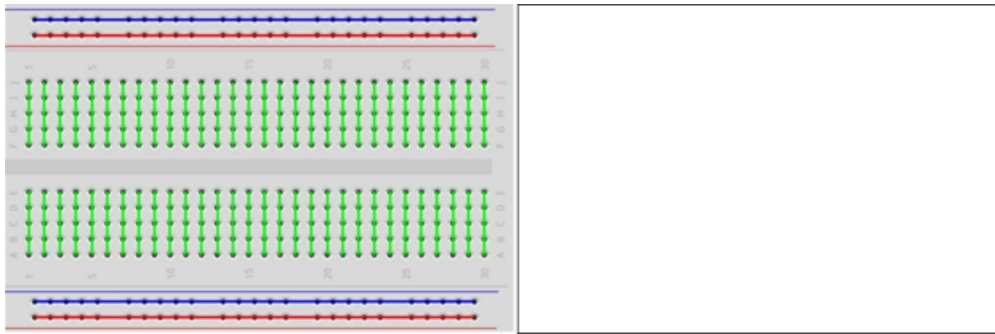
Conectores

Un “cable conector” es un tipo de cable diseñado para conectar componentes entre sí mediante la inserción de sus dos terminales. Los conectores tienen un extremo macho (pin) y un extremo hembra (ranura), por lo que se pueden clasificar en los tres tipos siguientes.



Placa de pruebas

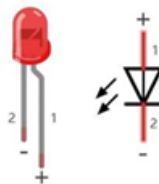
En la placa de pruebas hay muchos orificios pequeños para unir los conectores. Algunos de estos orificios están enlazados entre sí en el interior de la placa. Aquí tenemos una pequeña placa de pruebas a modo de ejemplo de cómo están conectadas eléctricamente las filas de orificios (zócalos). La imagen de la izquierda muestra cómo los pines comparten la conexión eléctrica, y la de la derecha muestra el metal interno que une eléctricamente estas filas.



Led (Externo)

Un LED es un tipo de diodo. Todos los diodos solo funcionan si la corriente circula en la dirección correcta y tienen dos polos. Un LED solo se iluminará si la patilla más larga (+) del LED se conecta al polo positivo de una fuente de alimentación y la patilla más corta se conecta al polo negativo (-) de la misma, también denominado tierra (GND). Este tipo de componente se conoce como “polar” (piensa en una calle de sentido único).

Todos los diodos comunes de dos terminales son iguales en este aspecto. Los diodos solo funcionan si la tensión de su electrodo positivo es superior a la de su electrodo negativo, y la mayoría de los diodos tienen un rango de tensión de funcionamiento muy reducido, comprendido entre 1,9 y 3,4 V. Si se aplica una tensión muy superior a 3,3 V, el LED se dañará y se fundirá.



Nota: Los LEDs no se pueden conectar directamente a una fuente de alimentación, ya que esto suele provocar daños en el componente. Es necesario conectar en serie una resistencia con un valor de resistencia específico al LED que se vaya a utilizar.

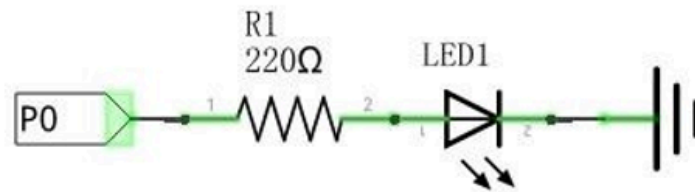
Esquema de conexión

Al realizar el cableado, se recomienda desconectar todas las fuentes de alimentación del circuito y, a continuación, montar el circuito siguiendo el esquema (la placa micro:bit no se puede insertar al revés), el polo positivo del LED (pin largo) debe conectarse a la resistencia, mientras que su polo negativo (pin corto) debe conectarse a GND. Una vez montado y comprobado que el circuito es correcto, utiliza el cable USB para conectar el ordenador al micro:bit y alimentar el circuito.

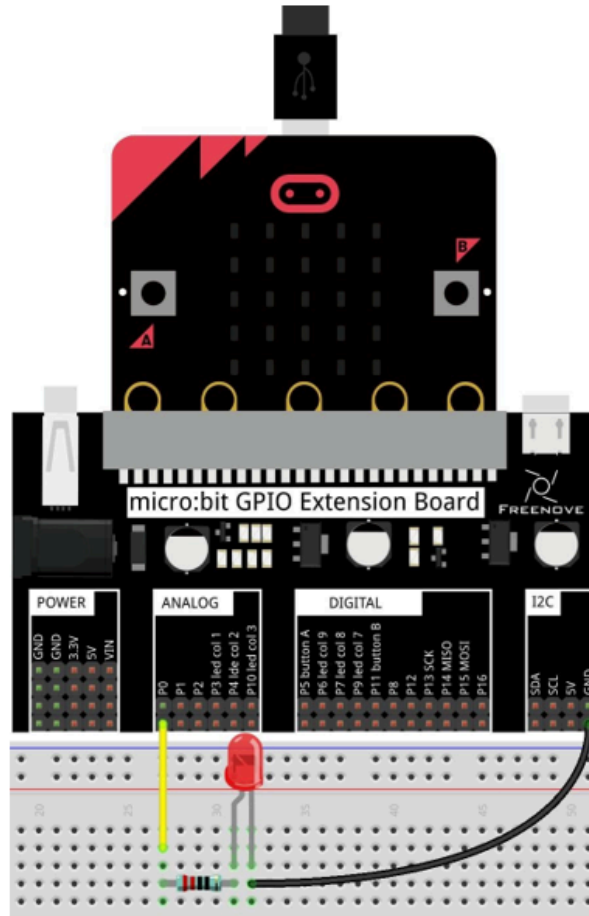
PRECAUCIÓN: ¡Evita cualquier posible cortocircuito (especialmente al conectar 5 V o GND, 3,3 V y GND)! **ADVERTENCIA:** ¡Un cortocircuito puede provocar una corriente elevada en tu circuito, generar un calor excesivo en los componentes y causar daños permanentes en tu micro:bit!

Nota: Patilla larga del diodo conectada en la misma columna que la resistencia, patilla corta del diodo conectada en la misma columna que GND.

Schematic diagram



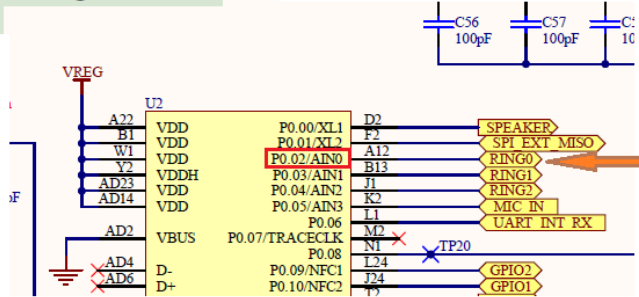
Hardware connection



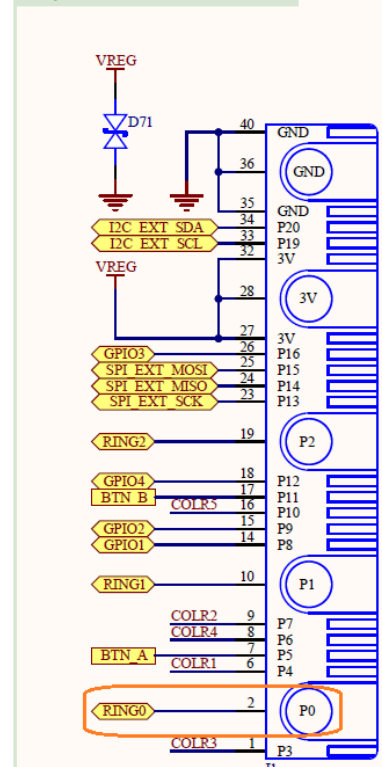
El puerto a usar sería el P0, el pin a utilizar el 02.

Veamos la siguiente imagen para entenderlo mejor. Si partimos de la placa de expansión (Expansion connector), queremos usar el conector **P0** (ver imagen anterior, cable amarillo) que se rotula como **RING0**. Esta marca nos lleva en la MCU al pin 02 del puerto P0. Por tanto, activando y desactivando este pin de la MCU lograremos encender/apagar el led de la placa (Board.edge.e00).

Target MCU



Expansion connector



Código fuente

```

#![no_main]
#![no_std]

use cortex_m_rt::entry;
use embedded_hal::delay::DelayNs;
use embedded_hal::digital::OutputPin;
use microbit::Board;
use nrf52833_hal::Timer;
use nrf52833_hal::gpio::Level;
use panic_rtt_target as _;
use rtt_target::{rprintln, rtt_init_print};

#[entry]
fn main() -> ! {
    rtt_init_print!();
    rprintln!("Iniciando programa");
    let board = Board::take().unwrap();
    let mut timer = Timer::new(board.TIMER0);
    let mut led = board.edge.e00.into_push_pull_output(Level::Low);

    loop {
        timer.delay_ms(1000_u32);
        rprintln!("Encendiendo");
        led.set_high().unwrap();
        timer.delay_ms(1000_u32);
        rprintln!("Apagando");
        led.set_low().unwrap();
    }
}

```

Explicación del código

Ahora que tenemos un poco de experiencia programando, ha llegado el momento de entender la estructura de las librerías en Rust, pero antes resolvemos una pequeña duda que nos puede surgir al leer el código fuente. Por qué: `board.edge.e00.into_push_pull_output(Level::Low);`. En la siguiente sección los explicaremos a fondo, pero por ahora nos basta con saber que esta línea de

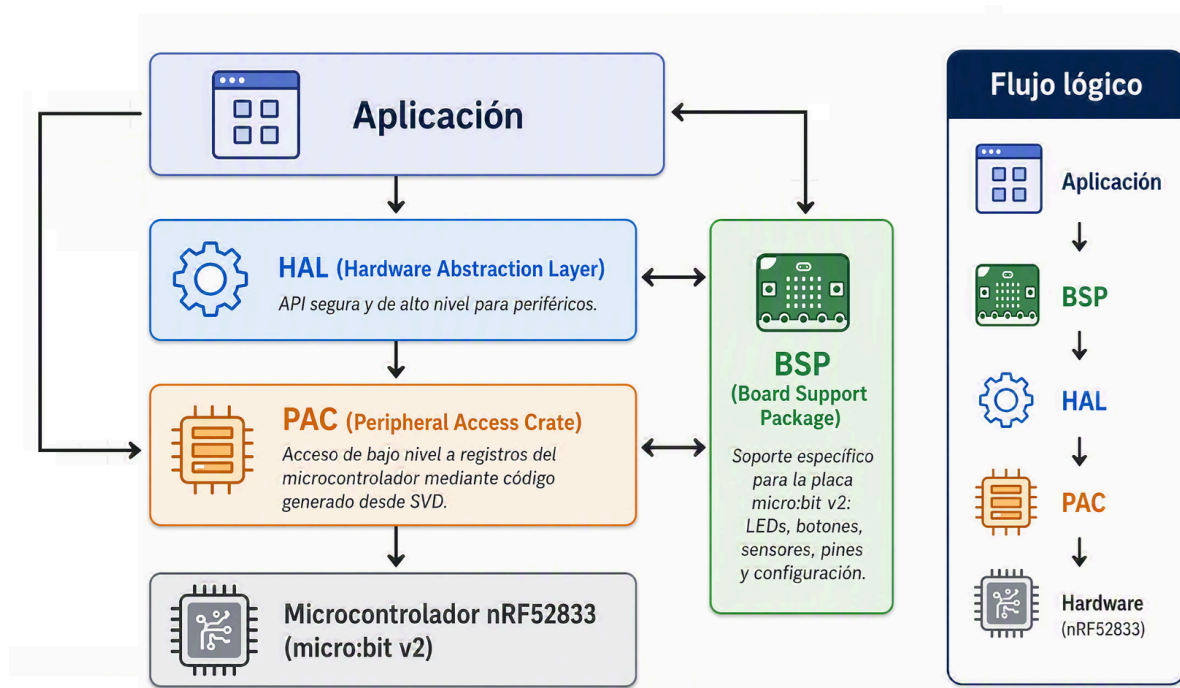
código configura el pin **P2** del puerto **P0** como salida digital y lo inicializa en nivel bajo (apagado), tal y como requiere el ejercicio.

Estructura de las librerías Rust

Estructura de las librerías Rust

Ahora que hemos avanzado en la comprensión del hardware y software, es el momento de profundizar en la estructura de las librerías Rust. En este apartado, se explicará la estructura de las librerías y cómo se relacionan entre sí.

En la imagen siguiente se puede ver un esquema de la estructura de las librerías Rust que vamos a usar para programar la micro:bit. En concreto siempre que podamos nos quedaremos al mayor nivel posible (BSP).



PAC

El trabajo del PAC es proporcionar una interfaz directa (más o menos segura) a los periféricos del chip, permitiendo configurar cada bit como queramos (por supuesto, también de forma incorrecta). Por lo general, solo habrá que lidiar con el PAC si las capas superiores no recogen todas las necesidades o cuando estemos desarrollando código de nivel superior para ellas. No es sorprendente que el PAC que vamos a usar (en su mayoría implícitamente) sea para el nRF52.

NOTA: El PAC actual está basado en la versión 1.3 de la especificación del producto, aunque la actual es la versión 1.7, por lo que podríamos encontrarnos con algunas diferencias. Sin embargo, la mayoría de las funciones que vamos a usar no han cambiado, por lo que no debería ser un problema.

HAL

La función del HAL es construir sobre el PAC del chip una capa y proporcionar una abstracción superior que sea realmente utilizable para alguien que no conoce todo el comportamiento del chip. Normalmente, la capa HAL abstrae los periféricos completos en estructuras individuales que pueden, por ejemplo, usarse para enviar datos a través del periférico. Vamos a usar el nRF52-hal.

BSP

La tarea del BSP es abstraer toda una placa (como la micro:bit) de una vez. Eso significa que tiene que proporcionar utilidades para usar tanto el microcontrolador como los sensores, LED, etc. que puedan estar presentes. Con bastante frecuencia (especialmente con placas hechas a medida) no existirá ningún BSP. En su lugar, trabajaremos con un HAL para el chip y construiremos los controladores para los sensores nosotros o los buscaremos en crates.io. Afortunadamente, la MB2 sí tiene un BSP, así que vamos a usarlo junto con el HAL.

El crate actual que vamos a usar es [microbit-common](#), que proporciona una abstracción de la placa MB2. La estructura principal de este crate es `board`, que representa la placa micro:bit y proporciona acceso a todos los periféricos y sensores de la placa. La estructura `Board` se puede acceder utilizando el método `Board::take()`, que devuelve una instancia de ella si no se ha creado ninguna otra previamente. Una vez que se tiene la instancia, se puede acceder a los periféricos y sensores de la placa a través de sus campos públicos.

Nota: La estructura `Board` es un singleton, lo que significa que solo puede haber una instancia en todo el programa. Esto se debe a que la placa MB2 tiene recursos limitados y no se pueden compartir entre múltiples instancias. Por lo tanto, si se intenta crear una segunda, se devolverá `None`.

```
let board = Board::take().unwrap();

let mut timer = Timer::new(board.TIMER0);
let mut led = board.edge.e00.into_push_pull_output(Level::Low);
```

Nota: el crate original que tenemos que usar es `microbit`, pero este lo único que hace es incluir el crate `microbit-common` y añadirle un par de cosas más, por lo que vamos a investigar directamente `microbit-common`.

```
use microbit::Board;
```

```
flowchart TD
    microbit_common --> GPIO
    microbit_common --> Display
    microbit_common --> ADC
    microbit_common --> Board
    GPIO --> DisplayPins
    GPIO --> MicrophonePins
    DisplayPins --> LEDMatrix[Matriz LED 5x5]
    Display --> blocking
    Display --> nonblocking

    style Board fill:#f00
```

Estructura Board

Se puede acceder a toda la documentación de la estructura en [Board](#)

Recursos de la placa

Nombre	Tipo	Descripción
<code>pins</code>	<code>Pins</code>	Pines GPIO que no están asignados a otros dispositivos de la placa.

Nombre	Tipo	Descripción
edge	Edge	Pines disponibles en el conector de borde de la micro:bit.
display_pins	DisplayPins	Pines utilizados por la matriz LED 5×5.
buttons	Buttons	Acceso a los botones de usuario de la placa.
speaker_pin	P0_00<Disconnected>	Pin asociado al altavoz integrado.
microphone_pins	MicrophonePins	Pines relacionados con el micrófono integrado.
i2c_internal	I2CInternalPins	Bus I²C utilizado internamente por la placa.
i2c_external	I2CExternalPins	Bus I²C accesible para dispositivos externos.
uart	UartPins	Pines UART conectados al depurador/interfaz USB.

Periféricos del núcleo ARM Cortex-M4

Nombre	Tipo	Descripción
CBP	CBP	Operaciones de mantenimiento de caché y predictor de saltos.
CPUID	CPUID	Información de identificación de la CPU.
DCB	DCB	Debug Control Block.
DWT	DWT	Data Watchpoint and Trace.
FPB	FPB	Flash Patch and Breakpoint.
FPU	FPU	Unidad de coma flotante.
ITM	ITM	Instrumentation Trace Macrocell.
MPU	MPU	Memory Protection Unit.
NVIC	NVIC	Controlador de interrupciones anidadas.
SCB	SCB	System Control Block.
SYST	SYST	Temporizador SysTick.
TPIU	TPIU	Trace Port Interface Unit.

Periféricos del nRF52833

Nombre	Tipo	Descripción
CLOCK	CLOCK	Control de relojes del sistema.
FICR	FICR	Factory Information Configuration Registers.
GPIOTE	GPIOTE	Eventos y tareas GPIO.
PPI	PPI	Interconexión directa entre periféricos.
PWM0	PWM0	Generador PWM 0.
PWM1	PWM1	Generador PWM 1.
PWM2	PWM2	Generador PWM 2.
PWM3	PWM3	Generador PWM 3.
RADIO	RADIO	Subsistema radio Bluetooth LE / 2.4 GHz.
RNG	RNG	Generador de números aleatorios hardware.
RTC0	RTC0	Real Time Counter 0.
RTC1	RTC1	Real Time Counter 1.
RTC2	RTC2	Real Time Counter 2.

Nombre	Tipo	Descripción
TEMP	TEMP	Sensor interno de temperatura.
TIMER0	TIMER0	Temporizador hardware 0.
TIMER1	TIMER1	Temporizador hardware 1.
TIMER2	TIMER2	Temporizador hardware 2.
TIMER3	TIMER3	Temporizador hardware 3.
TIMER4	TIMER4	Temporizador hardware 4.
TWIM0	TWIM0	Controlador I ² C maestro.
TWIS0	TWIS0	Controlador I ² C esclavo.
UARTE0	UARTE0	UART con EasyDMA.
UARTE1	UARTE1	UART con EasyDMA.
ADC	SAADC	Convertidor analógico-digital.
POWER	POWER	Gestión de energía.
SPI0	SPI0	Controlador SPI 0.
SPI1	SPI1	Controlador SPI 1.
SPI2	SPI2	Controlador SPI 2.
UART0	UART0	UART clásico.
TWI0	TWI0	I ² C clásico.
TWI1	TWI1	I ² C clásico.
SPIS1	SPIS1	SPI esclavo.
ECB	ECB	Cifrado AES ECB.
AAR	AAR	Address Resolution Accelerator.
CCM	CCM	Cifrado CCM para Bluetooth LE.
WDT	WDT	Watchdog Timer.
QDEC	QDEC	Decodificador de codificador rotatorio.
LPCOMP	LPCOMP	Comparador de baja potencia.
NVMC	NVMC	Controlador de memoria no volátil.
UICR	UICR	User Information Configuration Registers.

Métodos principales

Método	Descripción
<code>Board::take()</code>	Obtiene la instancia de la placa de forma segura. Solo puede ejecutarse una vez. Las llamadas posteriores devuelven <code>None</code> .
<code>Board::new(p, cp)</code>	Construye una instancia de <code>Board</code> a partir de unos <code>Peripherals</code> y <code>CorePeripherals</code> que ya hayan sido obtenidos previamente.

Resumen

microbit-common actúa como BSP (Board Support Package) para la BBC micro:bit v2. Reexporta HAL y PAC, proporciona abstracciones de placa, GPIO nombrados, soporte para la matriz LED y acceso simplificado a periféricos.


```

flowchart LR
    APP[Aplicación]
    BSP[microbit-common BSP]
    BOARD[Board]
    DISPLAY[Display]
    GPIO[GPIO]
    ADC[ADC]
    HAL[HAL]
    PAC[PAC]
    HW[nRF52833]
    APP --> BSP
    BSP --> BOARD
    BSP --> DISPLAY
    BSP --> GPIO
    BSP --> ADC
    BSP --> HAL
    HAL --> PAC
    PAC --> HW

```

Código de ejemplo

A modo de ejemplo hemos rehecho el mismo código que en el apartado anterior, pero usando solo el PAC. Veámoslo:

```

#![no_main]
#![no_std]

use cortex_m_rt::entry;
use embedded_hal::delay::DelayNs;
use embedded_hal::digital::OutputPin;
use nrf52833_hal::{gpio, pac, Timer};
use panic_rtt_target as _;
use rtt_target::{rprintln, rtt_init_print};

#[entry]
fn main() -> ! {
    rtt_init_print!();
    rprintln!("Iniciando programa");

    let peripherals = pac::Peripherals::take().unwrap();
    rprintln!("Periféricos adquiridos");
    let p0 = gpio::p0::Parts::new(peripherals.P0);
    let mut timer = Timer::new(peripherals.TIMER0);
    let mut led = p0.p0_02.into_push_pull_output(gpio::Level::Low);

    loop {
        timer.delay_ms(1000_u32);
        rprintln!("Encendiendo");
        led.set_high().unwrap();
        timer.delay_ms(1000_u32);
        rprintln!("Apagando");
        led.set_low().unwrap();
    }
}

```

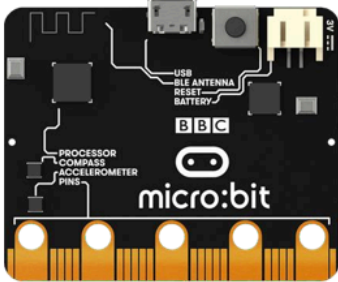
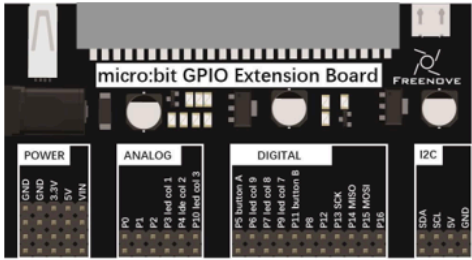
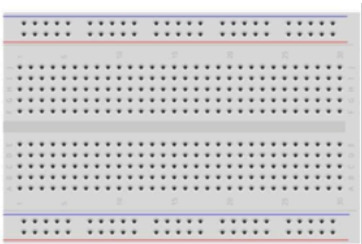
Capítulo 4 - Botones y Led

Por lo general, un dispositivo de control automático completo consta de tres partes esenciales: ENTRADA, SALIDA y CONTROL. En la sección anterior, el módulo LED era la parte de salida y el micro:bit, la parte de control. En aplicaciones prácticas, no solo hacemos que los LEDs parpadeen, sino que también conseguimos que un dispositivo detecte el entorno que le rodea, reciba instrucciones y, a continuación, realice la acción adecuada, como encender los LED, hacer sonar un zumbador, etc.

Descripción del proyecto 4.1

En este proyecto, controlaremos el estado del LED mediante un pulsador. Cuando se pulse el botón, el LED se encenderá, y cuando se suelte, el LED se apagará. Esto es lo que se conoce como un interruptor momentáneo. Utilizaremos un bucle de espere activa para programarlo.

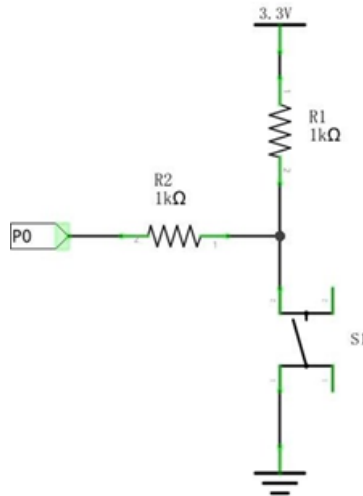
Hardware necesario

Microbit x1		Expansion board x1	
			
Breakboard x1		USB cable x1	
			
FM x4 MM x1	Resistor 220Ω x1	LED x1	Push Button Switch x1
			
	1kΩ x2		
			

Conociendo los componentes

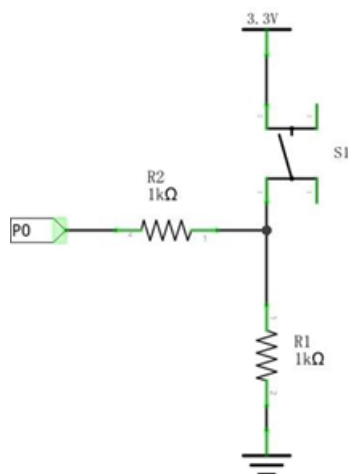
Conexión del interruptor de botón

Conectamos un interruptor de botón directamente al circuito para encender o apagar el LED. En los circuitos digitales, debemos utilizar el interruptor de botón como señal de entrada. La conexión recomendada es la siguiente:



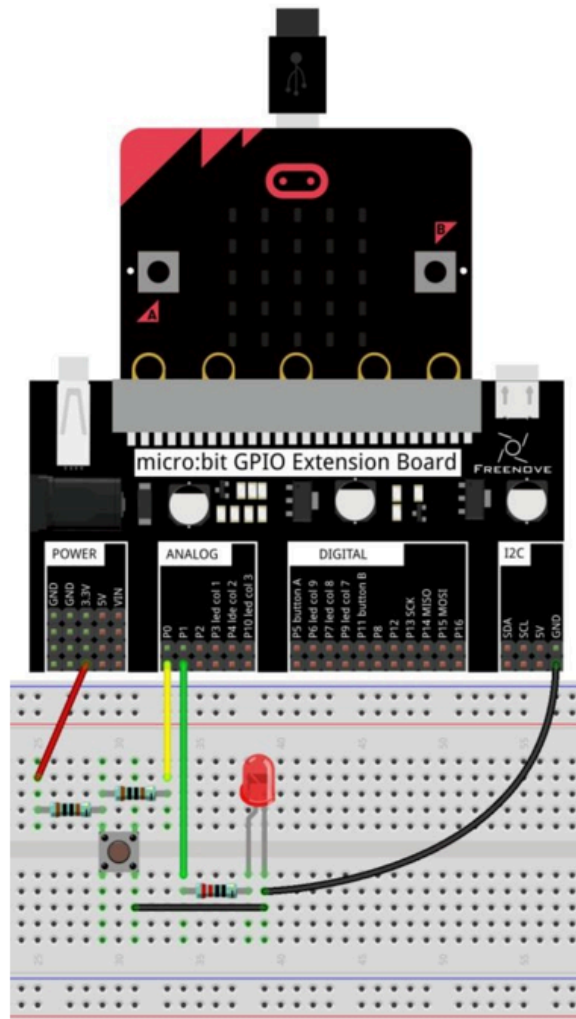
En el esquema de circuitos anterior, cuando no se pulsa el botón, el puerto de E/S detectará 3,3 V (nivel alto); y cuando se pulsa el botón, detectará 0 V (nivel bajo). La resistencia R2 se utiliza aquí para evitar que el puerto pase accidentalmente a un nivel alto de salida. Sin la R2, el puerto podría conectarse directamente al cátodo y provocar un cortocircuito al pulsar el botón.

El siguiente diagrama muestra otra conexión, en la que el nivel detectado por el puerto de E/S es el contrario al del diagrama anterior, independientemente de si se pulsa el botón o no.



Esquema de conexión

El pin P0 detecta el botón y el pin P1 controla el LED (P0_03 - RING1 - Board.edge.e01).



Código fuente

```
#![no_main]
#![no_std]

use cortex_m_rt::entry;
use embedded_hal::delay::DelayNs;
use embedded_hal::digital::{InputPin, OutputPin};
use microbit::Board;
use nrf52833_hal::Timer;
use nrf52833_hal::gpio::Level;
use panic_rtt_target as _;
use rtt_target::{rprintln, rtt_init_print};

#[entry]
fn main() -> ! {
    rtt_init_print!();
    rprintln!("Iniciando programa");
    let board = Board::take().unwrap();
    let mut timer = Timer::new(board.TIMER0);
    let mut led_p0 = board.edge.e00.into_pull_down_input();
    let mut led_p1 = board.edge.e01.into_push_pull_output(Level::Low);

    loop {
        timer.delay_ms(1000_u32);
        if led_p0.is_low().unwrap() {
            rprintln!("Encendiendo");
            led_p1.set_high().unwrap();
        } else {
            rprintln!("Apagando");
            led_p1.set_low().unwrap();
        }
        timer.delay_ms(1000_u32);
    }
}

cargo run
```

Explicación del código

El código es bastante sencillo, en este caso establecemos el P0 como entrada y el P1 como salida. Cuando se detecta que el botón está pulsado, se enciende el LED; cuando se detecta que el botón está suelto, se apaga el LED.

Descripción del proyecto 4.2

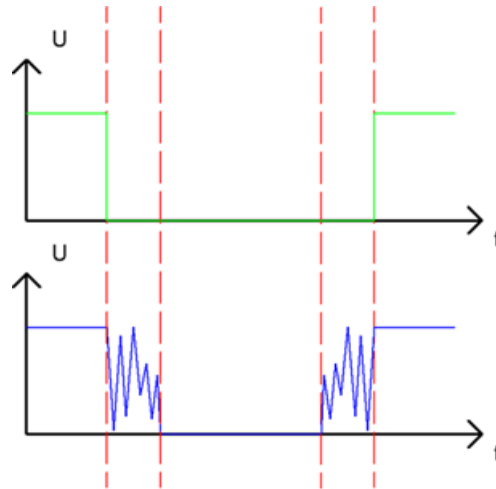
En este proyecto, vamos a fabricar una lámpara de mesa. Los componentes y circuitos utilizados son exactamente los mismos que en el proyecto anterior, pero esta funcionará de forma diferente: al pulsar el botón, el LED se encenderá, y al volver a pulsarlo, el LED se apagará. La acción del interruptor ya no es momentánea (como un timbre), sino que permanece encendido sin necesidad de mantener pulsado el interruptor de botón.

Hardware necesario

El mismo

Conociendo los componentes

Rebote del botón



Cuando se pulsa un interruptor de botón, no pasa de un estado a otro de forma inmediata. Debido a unas minúsculas vibraciones mecánicas, se produce un breve periodo de oscilación continua antes de que se establezca en el nuevo estado; este proceso es demasiado rápido para que los seres humanos lo detecten, pero no para los microcontroladores. Lo mismo ocurre cuando se suelta el interruptor del botón. Este fenómeno no deseado se conoce como “rebote”.

Por lo tanto, si procedemos a detectar directamente el estado del pulsador, se producen múltiples acciones de pulsación y liberación en un mismo ciclo de pulsación. Estas oscilaciones pueden confundir el funcionamiento a alta velocidad del microcontrolador y provocar numerosos errores. Por ello, debemos eliminar el impacto de dichas oscilaciones.

Nuestra solución: evaluar el estado del pulsador varias veces. Solo cuando el estado del pulsador sea estable (constante) durante un periodo de tiempo, ¿Puede indicar que el botón se encuentra realmente en el estado “ON” (pulsado)?

Este proyecto requiere los mismos componentes y circuitos que utilizamos en la sección anterior.

Esquema de conexión

El mismo que en la sección anterior.

Código fuente

```
#![no_main]
#![no_std]

use cortex_m_rt::entry;
use embedded_hal::delay::DelayNs;
use embedded_hal::digital::{InputPin, OutputPin};
use microbit::Board;
use nrf52833_hal::Timer;
use nrf52833_hal::gpio::Level;
use panic_rtt_target as _;
use rtt_target::{rprintln, rtt_init_print};

#[entry]
fn main() -> ! {
    rtt_init_print!();
    rprintln!("Iniciando programa");
    let board = Board::take().unwrap();
    let mut timer = Timer::new(board.TIMER0);
    let mut led_p0 = board.edge.e00.into_pull_down_input();
    let mut led_p1 = board.edge.e01.into_push_pull_output(Level::Low);
    let mut status:bool = true;

    loop {

        if led_p0.is_low().unwrap() {
            rprintln!("Pulsado");
            timer.delay_ms(100_u32);
            if led_p0.is_low().unwrap(){
                if status {
                    rprintln!("On");
                    status = false;
                    led_p1.set_high().unwrap();
                } else {
                    rprintln!("Off");
                    status = true;
                    led_p1.set_low().unwrap();
                }
            }
            rprintln!("fin if");
            while led_p0.is_low().unwrap(){
                rprintln!("while");
                timer.delay_ms(100_u32);
            }
        }
    }
}
```

```
cargo run --example lamp
```

Explicación del código

En el programa, cuando se detecta por primera vez que se ha pulsado el botón, se espera 10 ms para comprobar si se vuelve a pulsar el botón, con el fin de eliminar el efecto del rebote al pulsarlo. Y si el botón sigue pulsado por segunda vez, se considera que se ha pulsado y que se encuentra en un estado estable. De lo contrario, se considera que se trata de un rebote y se sale de esta comprobación.

Cuando se detecte que se ha pulsado, cambia el valor de "status". "Status" se utiliza para guardar el estado del LED. A continuación, escribe el nuevo valor en el puerto P1 para controlar el LED.

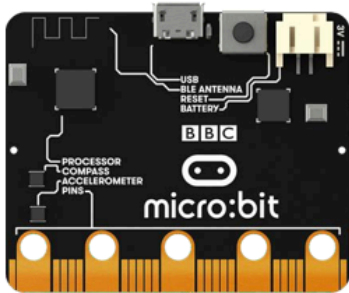
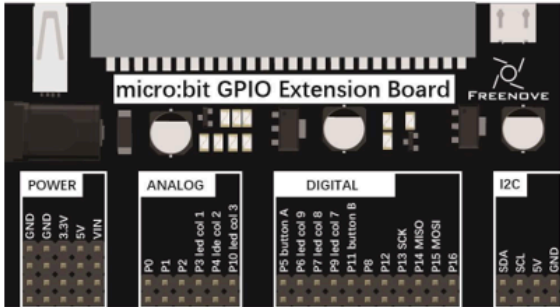
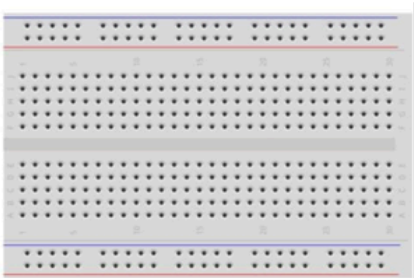
Una vez realizadas las operaciones anteriores, el programa detectará si se ha soltado el botón. Primero eliminará el rebote del botón mediante el while.

Capítulo 5 - Barra de Led

Descripción del proyecto 5.1

En este proyecto, utilizamos una barra LED para crear una luz que simula el agua fluyendo.

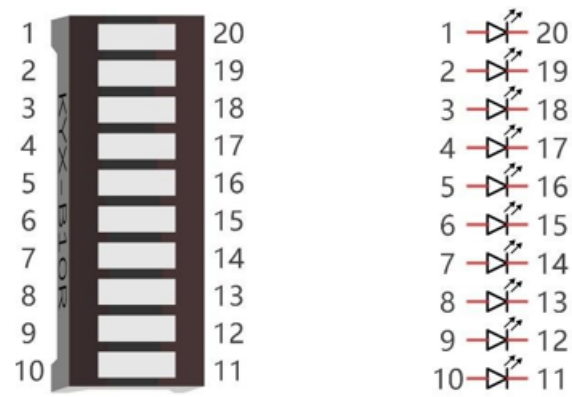
Hardware necesario

Microbit x1	Expansion board x1	
		
Breakboard x1	Resistor 220Ω x10	Jumper F/M x11
		
LED bar graph x1	USB cable x1	
		

Conociendo los componentes

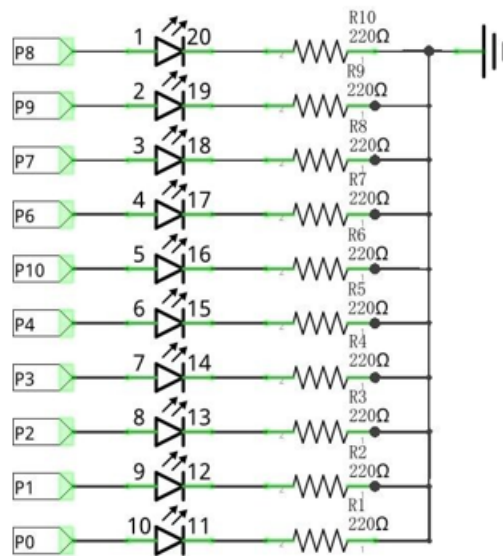
Barra Led

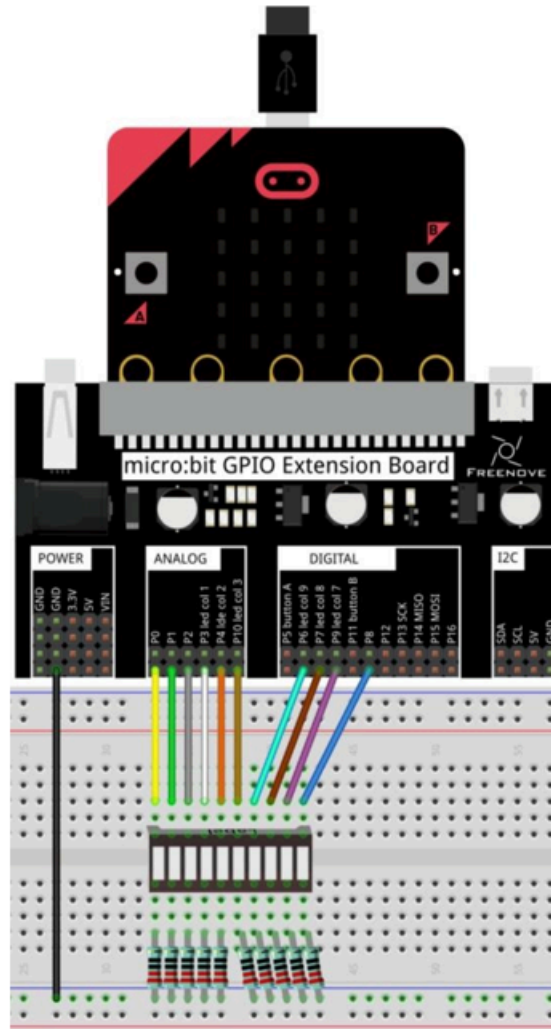
Una barra LED cuenta con 10 LEDs integrados en un único componente. Las dos filas de pines situadas en su parte inferior están emparejadas para identificar cada LED, al igual que el LED individual utilizado anteriormente.



Esquema de conexión

Diagrama esquemático





Nota: Si el proyecto no funciona, asegúrate de que la barra LED está conectada correctamente. Prueba a girar la barra led 180°.

Código fuente

Los pines a usar son: P0, P1, P2, P3, P4, P10, P6, P7, P9 y P8.

Se nombran como RING0, RING1, RING2, COLR3, COLR1, COLR5, COLR4, COLR2, GPIO2, GPIO1 respectivamente.

Se corresponden con: P0.02, P0.03, P0.04, P0.31, P0.28, P0.30, P1.05, P0.11, P0.09, P0.10.

En Rust: `board.edge.e00`, `board.edge.e01`, `board.edge.e02`, `board.display_pins.col3`, `board.display_pins.col1`, `board.display_pins.col5`, `board.display_pins.col4`, `board.display_pins.col2`, `board.edge.e09`, `board.edge.e08`

```

#![no_main]
#![no_std]

// Versión más fácil
use cortex_m_rt::entry;
use embedded_hal::delay::DelayNs;
use embedded_hal::digital::{OutputPin};
use microbit::Board;
use nrf52833_hal::Timer;
use nrf52833_hal::gpio::Level;
use panic_rtt_target as _;
use rtt_target::{rprintln, rtt_init_print};

#[entry]
fn main() -> ! {
    rtt_init_print!();
    rprintln!("Iniciando programa");

    let board = Board::take().unwrap();
    let mut timer = Timer::new(board.TIMER0);

    let mut led_p0 = board.edge.e00.into_push_pull_output(Level::Low);
    let mut led_p1 = board.edge.e01.into_push_pull_output(Level::Low);
    let mut led_p2 = board.edge.e02.into_push_pull_output(Level::Low);

    let mut led_p3 = board.display_pins.col3.into_push_pull_output(Level::Low);
    let mut led_p4 = board.display_pins.col1.into_push_pull_output(Level::Low);
    let mut led_p10 = board.display_pins.col5.into_push_pull_output(Level::Low);
    let mut led_p6 = board.display_pins.col4.into_push_pull_output(Level::Low);
    let mut led_p7 = board.display_pins.col2.into_push_pull_output(Level::Low);

    let mut led_p9 = board.edge.e09.into_push_pull_output(Level::Low);
    let mut led_p8 = board.edge.e08.into_push_pull_output(Level::Low);

    loop {
        led_p0.set_high().unwrap();
        timer.delay_ms(500_u32);
        led_p0.set_low().unwrap();
        led_p1.set_high().unwrap();
        timer.delay_ms(500_u32);
        led_p1.set_low().unwrap();
        led_p2.set_high().unwrap();
        timer.delay_ms(500_u32);
        led_p2.set_low().unwrap();
        led_p3.set_high().unwrap();
        timer.delay_ms(500_u32);
        led_p3.set_low().unwrap();
        led_p4.set_high().unwrap();
        timer.delay_ms(500_u32);
        led_p4.set_low().unwrap();
        led_p10.set_high().unwrap();
        timer.delay_ms(500_u32);
        led_p10.set_low().unwrap();
        led_p6.set_high().unwrap();
        timer.delay_ms(500_u32);
        led_p6.set_low().unwrap();
        led_p7.set_high().unwrap();
        timer.delay_ms(500_u32);
        led_p7.set_low().unwrap();
        led_p9.set_high().unwrap();
        timer.delay_ms(500_u32);
        led_p9.set_low().unwrap();
        led_p8.set_high().unwrap();
        timer.delay_ms(500_u32);
        led_p8.set_low().unwrap();
    }
}

```

cargo run

Explicación del código

Este código es sencillo, enciende de forma secuencialy apaga cada uno de los pines necesarios dentro de un bucle infinito, usando un retardo de 500 milisegundos.

Código fuente (versión dos)

```
#![no_main]
#![no_std]

use cortex_m_rt::entry;
use embedded_hal::delay::DelayNs;
use embedded_hal::digital::{StatefulOutputPin};
use microbit::Board;
use nrf52833_hal::Timer;
use nrf52833_hal::gpio::{Level, Output, Pin, PushPull};
use panic_rtt_target as _;
use rtt_target::{rprintln, rtt_init_print};

#[entry]
fn main() -> ! {
    rtt_init_print!();
    rprintln!("Iniciando programa");
    let board = Board::take().unwrap();
    let mut leds:[Pin<Output<PushPull>>; 10] = [
        board.edge.e00.degrade().into_push_pull_output(Level::Low),
        board.edge.e01.degrade().into_push_pull_output(Level::Low),
        board.edge.e02.degrade().into_push_pull_output(Level::Low),
        board.display_pins.col3.degrade().into_push_pull_output(Level::Low),
        board.display_pins.col1.degrade().into_push_pull_output(Level::Low),
        board.display_pins.col5.degrade().into_push_pull_output(Level::Low),
        board.display_pins.col4.degrade().into_push_pull_output(Level::Low),
        board.display_pins.col2.degrade().into_push_pull_output(Level::Low),
        board.edge.e09.degrade().into_push_pull_output(Level::Low),
        board.edge.e08.degrade().into_push_pull_output(Level::Low)
    ];
    let mut timer = Timer::new(board.TIMER0);

    loop {
        for led in leds.iter_mut() {
            led.toggle().ok();
            timer.delay_ms(500u32);
            led.toggle().ok();
        }
    }
}
```

```
cargo run --example main_for
```

Explicación del código

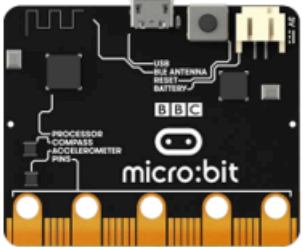
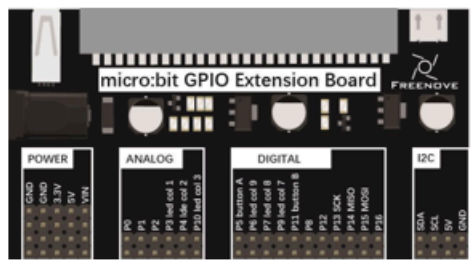
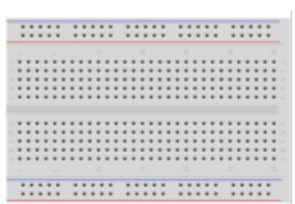
En este caso vamos a usar un Array para almacenar los pines de la barra led, y un bucle for para recorrerlos. Esto nos permite encender y apagar los leds de manera más eficiente y con menos código.

Capítulo 6 - PWM

Descripción del proyecto

Vamos a aprender a hacer parpadear un led de manera gradual, es decir, que vaya aumentando y disminuyendo su brillo. Para ello vamos a utilizar la técnica de modulación por ancho de pulso (PWM).

Hardware necesario

Microbit x1 	Expansion board x1 	
Breakboard x1 	USB cable x1 	
Jumper F/M x2 	Resistor 220Ω x1 	LED x1 

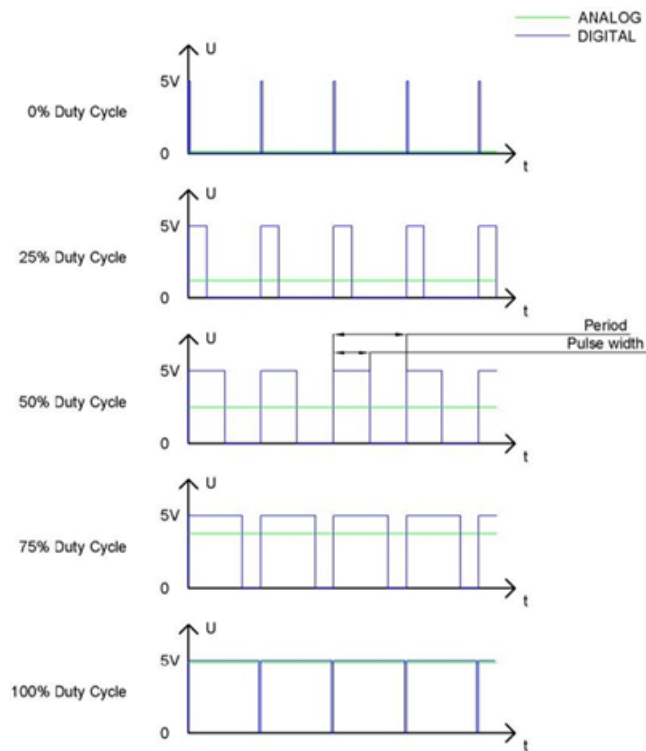
Conociendo los componentes

PWM

La PWM (modulación por ancho de pulso) es un método muy eficaz para utilizar señales digitales con el fin de controlar circuitos analógicos. Los procesadores digitales no pueden generar directamente señales analógicas. La tecnología PWM facilita enormemente esta conversión (la transformación de señales digitales en analógicas).

La tecnología PWM utiliza pines digitales para enviar ondas cuadradas de determinadas frecuencias, es decir, señales de nivel alto y nivel bajo que se alternan durante un tiempo determinado. El tiempo total de cada conjunto de niveles altos y bajos suele ser fijo, lo que se denomina «período» (Nota: el recíproco del período es la frecuencia). El tiempo de las salidas de nivel alto se denomina generalmente «anchura de pulso», y el ciclo de trabajo es el porcentaje que representa la relación entre la duración del pulso, o anchura de pulso (PW), y el período total (T) de la forma de onda.

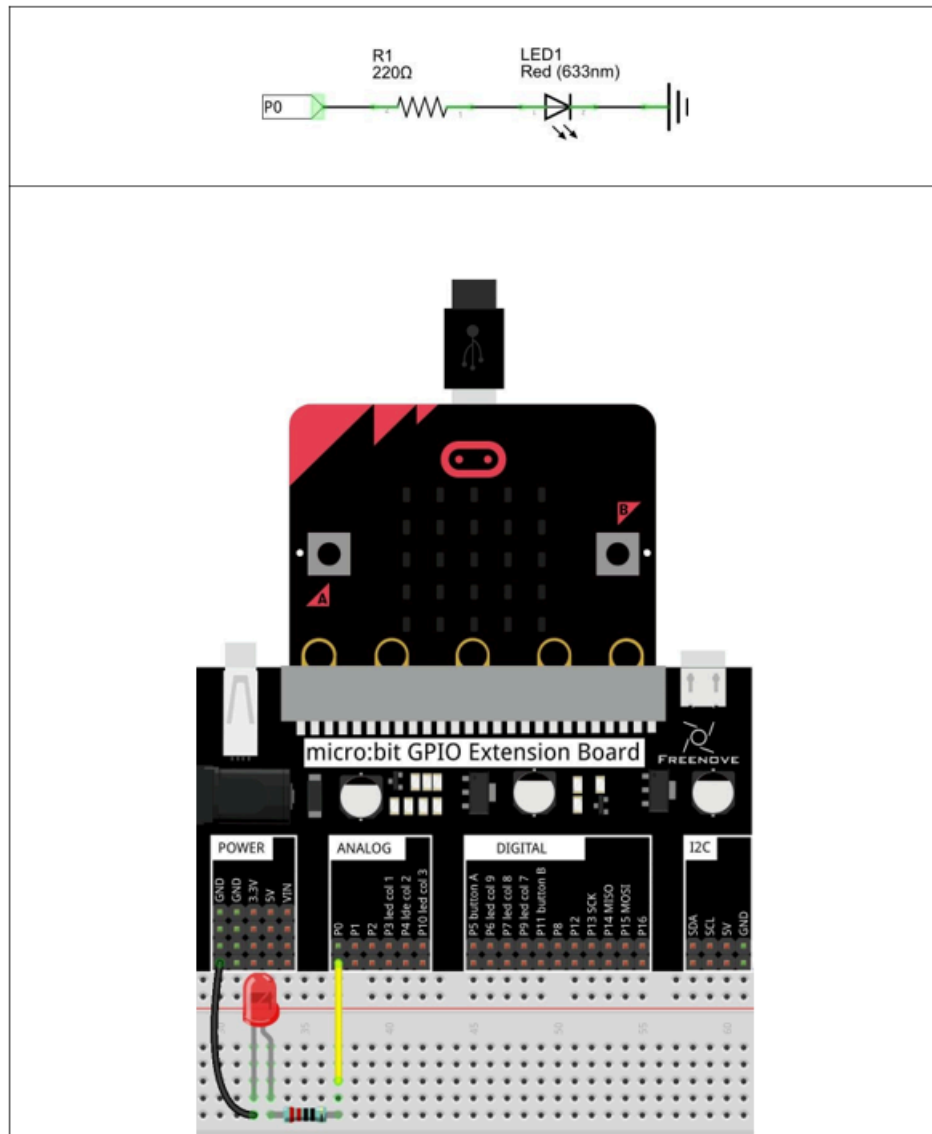
Cuanto más dure la salida de los niveles altos, mayor será el ciclo de trabajo y mayor será la tensión correspondiente en la señal analógica. Las siguientes figuras muestran cómo varían las tensiones de la señal analógica entre 0 V y 5 V (el nivel alto es 5 V) en función de la anchura del pulso del 0 % al 100 %:



Cuanto más largo sea el ciclo de trabajo del PWM, mayor será la potencia de salida. Ahora que comprendemos esta relación, podemos utilizar el PWM para controlar el brillo de un LED o la velocidad de un motor de corriente continua, entre otras cosas.

Esquema de conexión

Diagrama esquemático



Nota: Patilla larga del diodo conectada en la misma columna que la resistencia, patilla corta del diodo conectada en la misma columna que GND.

Código fuente

El pin a usar es: P0.

Se nombra como RING0.

Se corresponde con: P0.02.

En Rust: board.edge.e00.


```

#![no_main]
#![no_std]

use embedded_hal::delay::DelayNs;
use nrf52833_hal::{pwm, Timer};
use cortex_m_rt::entry;
use embedded_hal::digital::OutputPin;
use microbit::Board;
use panic_halt as _;
use nrf52833_hal::gpio::{Level, Pin};
use nrf52833_hal::pwm::{Channel, Pwm};

#[entry]
fn main() -> ! {
    let board = Board::take().unwrap();
    let mut timer = Timer::new(board.TIMER0);
    let mut led = board.edge.e00.into_push_pull_output(Level::Low);
    let duty_values = [ 0, 4000, 8000, 12000, 16000, 20000, 24000, 28000, 32000 ];

    led.set_high().unwrap();
    timer.delay_ms(1000_u32);

    let pwm = Pwm::new(board.PWM0);
    pwm.set_output_pin(Channel::C0, Pin::from(led)); // Asignar el Pin P0.02 - RING0 -
P0 al canal 0
    let max_duty = pwm.max_duty();

    loop {
        // Efecto "Fade In": Incrementa el brillo
        for duty in duty_values {
            pwm.set_duty_on_common(duty);
            timer.delay_ms(200_u32);
        }

        pwm.set_duty_on_common(max_duty);
        timer.delay_ms(500_u32);

        // Efecto "Fade Out": Decrementa el brillo
        for duty in duty_values.into_iter().rev() {
            pwm.set_duty_on_common(duty);
            timer.delay_ms(200_u32);
        }
    }
}

cargo run

```

Explicación del código

Definimos un array con los valores de ciclo de trabajo que queremos usar para el PWM. En este caso, vamos a hacer que el LED vaya aumentando y disminuyendo su brillo de manera gradual. Para ello, definimos un array con valores que van desde 0 hasta 32000 (el valor máximo para un PWM de 15 bits es 32767) y luego volvemos a 0.

```
let duty_values = [0, ...
```

El código a partir de ahí es sencillo, inicializamos el módulo PWM. Unimos el canal 0 del mismo al pin P0 del conector externo (P0.02 o RING0) correspondiente, después con dos bucles uno para incrementar el brillo y otro para decrementarlo, establecemos el valor del ciclo y esperamos un pequeño tiempo para que no sea muy rápido el cambio. Entre cada bucle aguardaremos un tiempo mayor a la máxima potencia para que la visualización del cambio de brillo sea más clara.